

Defense guide: prefix-cost-attribution

For reading before an interview. Every number in this file is re-measured by `tools/collect_metrics.py` and checked against this text by `tools/check_numbers.py`, so a sentence that has gone stale fails the build rather than getting read aloud.

Read it in three passes. The first two sections are what to say in the first minute. The claims table and the sections under it are what to say when someone picks one. The last three sections are what to say when someone attacks it, which is the part worth rehearsing.

The thirty second version

"A prefix cache means the server does not process most of the prompt tokens it is sent. On this corpus it processes about seven percent of them. So a bill that charges per token sent is not a division of what the server spent: it **collects 1.8378 times** the real cost, and **13.7723 times the fair share** on the prefill line. Charge for the tokens actually processed instead and the total is exactly right and the split becomes a record of arrival order: across every prompt family in every replay, that scheme puts the **first arriver top in 360 of of 360 family replays**, where **chance would be 0.25.**"

Then stop. The question that usually comes next is "so which scheme should I use", and the answer is a boundary rather than a scheme, which is the least convenient answer available and the reason the tool has three verdicts.

The two minute version

The structural claim first, because it does not depend on my corpus. Prefill work under a cache that never evicts is the number of nodes in the trie of all request token sequences. That is an integer, it is computable without simulating anything, and it is the same integer under every permutation of the arrival order. With no cache it is the number of prompt tokens sent. Both ends are exact, which means everything measured between them is calibrated against something rather than against itself.

Then the measurement, because a structural claim does not tell anyone what it costs them. **24 tenants** across **6 prompt families** send **960 requests** carrying **337,098 prompt tokens**. At the shipped **capacity of 8,000 tokens** the server **actually processes 25806** of them, a **hit share of 0.9234**. A per request bill therefore **collects 1.8378 times** the spend, which is **0.8379 more than the server** paid, and the **worst line is 15.1728 times** a tenant's fair share of prefill.

Then the part that makes it a design decision rather than an observation. Three properties are worth wanting: sum to the spend, do not move with arrival order, split shared work fairly. Across **24 configurations**, no scheme has all three in any of the **13 of them recompute** an evicted prefix. The exact Shapley value has **all three in 11 of** the other eleven and **has all three in 0 of the** recomputing ones. So the conflict is created by recomputation, not by the choice of scheme, and that is an actionable statement where an impossibility theorem would not have been.

The claims, and how each one is proved

Claim	Command	The number that settles it
Two quantities here are exact, not estimated	<code>python experiments/exp01_the_two_exact_anchors.py</code>	prefill equals the trie node count under every permutation tried
The efficient bill is a record of arrival order	<code>python experiments/exp02_the_same_usage_two_bills.py</code>	a median tenant spread of 0.3494 against exactly 0.0 for the others
The capacity is fine and the policy is the cost	<code>python experiments/exp03_the_policy_not_the_capacity.py</code>	an optimal policy reaches the unbounded optimum at a third of the working set
The first arriver pays, every single time	<code>python experiments/exp04_who_pays_the_surplus.py</code>	360 of 360 family replays against a chance rate of one in four
No scheme has all three properties when the cache recomputes	<code>python experiments/exp05_no_scheme_has_all_three.py</code>	0 of 13 recomputing cells, 11 of 11 otherwise
The exact allocation is cheaper than a sampled one and has no error	<code>make bench</code>	261.6 times faster, and the sampled one is still wrong

Two quantities here are exact, not estimated

With no cache, prefill equals the **337,098 prompt tokens** sent. With a cache large enough never to evict, it equals the **24,613 distinct prefix tokens** in the trie, and that holds **under 96 permutations** of the arrival order as well as on the identity order. Both are asserted with == on integers rather than within a tolerance.

If pushed on "why does that matter": because every other number in the repository is a share of prefill, and a share needs a denominator you can check independently. When the simulator was wrong, this is what caught it: the unbounded case did not land on the trie's node count, and the gap was a quarter of the reuse.

The efficient bill is a record of arrival order

Over **120 tenant observations** across **12 arrival orders** of the same requests, the median tenant's share under marginal **moves by 0.3494 of its own value** and the **worst tenant moves 0.4975**. One tenant was **charged 813 in one arrival order and 1,336 in another**. Every order independent scheme moved by exactly nothing: **120 of them exactly** unchanged, to the last bit.

If pushed on "0.3494 sounds small for a bill": it is a third of a tenant's prefill line moving because of who else was scheduled first, and the customer receiving it has no way to see that, since the requests they sent were identical. The version of this a support ticket reaches you as is "my usage was the same and my bill moved".

The capacity is fine and the policy is the cost

At the shipped capacity **an oracle needs only 24,613 prefill tokens**, which is the unbounded optimum exactly: an optimal policy at **0.33 of the working set** never evicts anything that will be needed again. LRU at the same capacity is **spending 0.0532 more than** it needs to, and across the sweep the policy costs as much as **as much as 0.2995 more** than optimal.

If pushed on "an oracle is not implementable, so what": it is not a recommendation, it is a decomposition. A single prefill number cannot tell "the cache is too small" from "the eviction policy is wrong", which are different

complaints with different fixes and very different costs. The oracle is what separates them.

The first arriver pays, every single time

Tenants sharing a prompt family share its opening tokens, so in any given replay exactly one of them pays to compute them. In a family of four, chance alone would put that tenant top a quarter of the time. The efficient scheme puts the **first arriver top in 360 of 360 family replays**; the **fair split does it 0.2167** of the time, which the interval around it does not distinguish from chance. Per replay, the **rank correlation of -0.2338** with arrival position, **against 0.0037 for the fair split**.

If pushed on "your correlation is only a fifth": that is why the family test is there. A rank correlation over twenty four tenants is a blunt instrument, and the family question has an exact null and an unambiguous answer. Lead with 360 of 360 and quote the correlation second.

No scheme has all three properties when the cache recomputes

Across a grid of tenant counts, prompt family counts and capacities, **24 configurations** in total, **13 of them recompute** a prefix the cache had already computed. In those the fair split **has all three in 0 of the cells** and so does everything else; in the other eleven it has **all three in 11 of them**.

If pushed on "that is just your grid": the grid is small on purpose because the three properties are checked exactly rather than statistically, so nothing about them needs a large sample. The claim that would need a bigger grid is a magnitude, and no magnitude is claimed here.

The exact allocation, if anyone asks

Measured **from 120 requests out to 960 requests**, over **7 timed repeats** on a two vCPU container running **Python 3.11.15**. The **exact pass takes 1.8 ms** at the smallest size and **9.2 ms at the largest**, where the **sampled pass takes 2411.3 ms at 200 permutations**, which is **261.6 times the exact pass**, with a **linearity ratio of 1.01**.

The point of that section is one sentence: on a tree the Shapley value is one pass, so the allocation everyone dismisses as exponential is the cheap one here. The accuracy half matters more. The smallest sample that keeps every tenant inside materiality **takes 200 permutations** and is **still wrong by 0.0381**; at ten it is **off by 0.1897 at ten**. A sampled allocation hands a customer an invoice with an error bar on it.

Questions that are meant to be hard

Is this just Shapley value with extra steps? The Shapley value is textbook and the tree shortcut is known in the cost allocation literature. What is mine: noticing that a prefix cache turns LLM billing into a cost game on a tree so the shortcut applies, the two exact anchors that make every share checkable against an integer, the measurement that the only efficient scheme charges the first arriver in 360 of 360 family replays, the boundary result that recomputation rather than eviction is what breaks efficiency, the three verdicts with not-an-attribution outranking the rest, and the receipts pipeline that fails the build when a document quotes a number the code no longer produces. Around a thousand statements of source, **105 tests, 99.9 percent line coverage**.

Your corpus is generated. Does any of it transfer? Two kinds of claim, and they transfer differently. That prefill under an unbounded cache equals the trie node count is arithmetic and holds on any workload. That the efficient scheme is order dependent is structural: it charges whoever met a cold cache, and that is true of any prefix cache. The magnitudes do not transfer: a **hit share of 0.9234** is a property of six prompt families with a long shared preamble, and the policy file exists so a team can put their own shape in.

What is the weakest part? The workload generator. Conversations are independent, arrivals are permuted as blocks, and there is no diurnal pattern or burstiness. Real traffic is bursty and correlated, and burstiness would concentrate cold cache misses on whoever happens to open a burst, which probably makes the order dependence worse rather than better, but I have not measured that and will not claim it. Second weakest: the cache is token granular where a real one is block granular, which is in ADR-001 as a named omission rather than an oversight.

Did anything go wrong while you built it? Three things, all in the README under "Hardest problem solved". The one worth telling: my first cache stored whole prompts, which is a request cache wearing a prefix cache's name. It reported 28,504 prefill tokens under an unbounded cache where the trie says **24,613 distinct prefix tokens**. What surfaced it was not a failing test, it was that the unbounded case did not land on an integer I could compute independently.

Why does your recommended scheme not return exit 0? Because at the shipped capacity it does not sum to what the server spent. The **fair split collects 0.9964, short of the bill by 0.0036**, and the provider absorbs the rest. I could have widened the tolerance until it passed, which would be setting a threshold from the answer. Instead the shortfall is printed as a magnitude next to the boolean, and ADR-004 records why not - an - attribution outranks everything else.

Is the surplus not just margin? Providers are allowed to make money. Yes, and that is the honest framing. Nothing here says a provider must pass the cache saving on. What it says is that the resulting number is not a division of anything, so it cannot be explained to a customer as one, and a company that describes its bill as usage based should know which of those it is doing. Choosing what to charge is explicitly out of scope.

Would this have caught a real incident? It would catch the class where a multi tenant deployment charges tenants inconsistently for shared prefill. It would not catch a pricing bug, a metering bug, or a tenant charged for another tenant's requests, and it cannot tell you what you should charge. Both limits are in the repository rather than in this answer.

What this repository cannot establish

Its own section, because it is the part a senior reviewer reads first, and offering it before being asked is worth more than any of the claims above.

- **The corpus is generated.** The identities transfer because they are arithmetic. The magnitudes are properties of the workload parameters in the policy file.
- **The tokeniser is trained here rather than downloaded.** That makes every count reproducible offline and makes the absolute counts larger than a production vocabulary would give. Every conclusion is a ratio, so the ratios are unaffected, and ADR-003 states the trade.
- **The cache is token granular.** Real caches evict blocks. Modelling that rounds prefixes to block boundaries and, on the evidence in the sweep, changes no conclusion, which is a belief rather than a measurement.
- **Arrivals are permuted as blocks with turn order preserved.** No burstiness, no diurnal pattern, no correlation between tenants.
- **Prices are chosen for legibility.** The absolute totals are not comparable to any invoice. Only the ratios are.
- **Whether the provider should pass the saving on is not answered.** That is a commercial decision this repository can measure the consequences of and cannot make.

Things to say, and things not to say

Say:

- "the server processes about seven percent of the prompt tokens it is sent, and the bill charges for all of them."
- "360 of 360 family replays, against a chance rate of one in four" rather than a correlation.
- "the conflict is created by recomputation, not by the choice of scheme."
- "on a tree the Shapley value is one pass, so the expensive allocation is the cheap one here."
- "I do not know what a provider should charge. This measures the consequences of the choice."

Do not say:

- **"providers are overcharging."** Say the bill is not a division of what the server spent, which is a statement about arithmetic rather than about intent.
- **"the marginal scheme is unfair."** It is efficient and order dependent, which are both precise, and unfair is the word for a different property that `equal_split` also lacks.
- **"Shapley is exponential."** In general yes, on a tree no, and the whole technical point is that this game is on a tree.
- **"99.9 percent coverage means it is correct."** It means the lines ran. The test that means something is the one that checks the one pass formula against the definition over all 120 permutations of five players.
- **"this is what your cloud bill does."** Say it is what a per token bill does once a prefix cache is switched on, and that whether any given provider bills that way is a question for their price sheet.

The live demo, four commands

```
# 1. What this configuration will build, and what each scheme is.
python -m prefixcost plan

# 2. A per request token bill, audited. Exit 2: it is not a division of the spend.
python -m prefixcost audit --seed 11 --scheme per_request

# 3. Charge for what was processed. Exit 1: the shares move with arrival order.
python -m prefixcost audit --seed 11 --scheme marginal

# 4. What the cache buys by capacity, against a policy that cannot be implemented.
python -m prefixcost cache --seed 11 --oracle
```

make bench and make charts regenerate the cost table and its chart from the same JSON, which is the shortest demonstration that no figure in either document was typed by hand.

Where to look in the code

Question	File
The two exact anchors, and why the cache is the trie	<code>src/prefixcost/trie.py</code> , <code>src/prefixcost/serving.py</code>
The tokeniser, and the two details that are load bearing	<code>src/prefixcost/tokenizer.py</code>
The five schemes, and the one pass Shapley value	<code>src/prefixcost/attribution.py</code>
Leaf inwards eviction, and the oracle that bounds LRU	<code>src/prefixcost/serving.py</code> , <code>_lru_victim</code> and <code>_oracle_victim</code>
The three verdicts and the ordering rule between them	<code>src/prefixcost/audit.py</code> , <code>decide_verdict</code>
The identity checked against the definition	<code>tests/test_shapley_against_brute_force.py</code>
Every price and threshold, with a comment on who chooses it	<code>configs/policy.yaml</code>
The five decisions and their rejected alternatives	<code>docs/adr/</code>